

Workshop · Module 05

Async *patterns.*

05 / 08

Agenda

ARC 1

THE SURFACE

Submit, then poll

- o1 **Async training — submit_fit_task**
Same arguments as fit(). Returns in about a second with a task id.
- o2 **Async prediction — two submits, one poll**
submit_predict_task for labels,
submit_predict_proba_task for scores. Both share
poll_predict_result.
- o3 **Matching polls — fit, predict, proba, feature importance**
Four submit verbs, three poll methods — predict and
predict_proba share poll_predict_result.

ARC 2

THE PAYOFFS

Parallel, portable, persistent

- o1 **Parallel jobs — one shared poll loop**
Submit N before polling any. Wall-clock is the longest job,
not the sum.
- o2 **48-hour task ids — portable strings**
Save the id, poll from a different session, a different script,
a different machine.
- o3 **Exercise — slim vs full, in parallel**
Two jobs, one wait. Compare AUC. Write the fallback note.



By the end of this module, you'll be able to...

Three things you'll *leave with*

A

Train and predict *without blocking*

submit_fit_task returns a task id in about a second. poll_fit_result returns Self when the model is ready. predict and predict_proba have matching submit/poll pairs.

B

Run N jobs *in parallel*

Submit multiple experiments before polling any of them. One shared loop checks each. Wall-clock time equals the longest job, not the sum.

C

Resume work *across sessions*

Task ids are strings, valid for 48 hours. Save them to disk or a database, poll from any new NEXUSClassifier instance — no shared state required.



*Submit.
Don't wait.*

PART 01



THE PAIN

Sync blocks. Async *frees* your process.

Call sync *fit()* and your Python process is tied up for minutes. If the network drops, you lose the job. You cannot run two experiments in parallel. You cannot kick off training before a meeting and score the model after.

Every production integration hits these walls eventually. Async is how you walk through them instead.

WHAT ASYNC DELIVERS

*Fire and
forget.
Check back
when ready.*

submit_fit_task returns in about a second with a task id. Training runs on Fundamental's infrastructure. Your process is free — run other jobs in between, or close the laptop and come back tomorrow.

*seconds →
minutes*

submit_fit_task
returns in about
a second — the
training minutes
are not yours to wait.



The one-sentence definition.

A task id is a string — save it,
pass it, poll it, for 48 hours, from
any session.

FIG. 05 · SUBMIT IS THE ORDER. TASK_ID IS THE TICKET. POLL IS "COME BACK WHEN IT'S READY."



Async training.

submit returns in about a second. poll returns Self when done.

● ● ● module_05_async_patterns.ipynb · cells 8 & 10

```
nexus_async = NEXUSClassifier(mode="quality")
```

```
# submit – returns a task id immediately
```

```
fit_task_id = nexus_async.submit_fit_task(  
    X_train_slim, y_train,  
)
```

```
print(f"submitted: {fit_task_id}") # save this
```

```
# poll – returns Self when done, None if still running
```

```
POLL_INTERVAL = 20
```

```
fitted = nexus_async.poll_fit_result(fit_task_id)
```

```
while fitted is None:
```

```
    time.sleep(POLL_INTERVAL)
```

```
    fitted = nexus_async.poll_fit_result(fit_task_id)
```

```
# nexus_async is now fitted – ready for predict, proba, feature importance
```

```
print(nexus_async.trained_model_id_)
```

WHY THIS MATTERS

Your process is free immediately.

The training job runs on Fundamental's infrastructure. submit returns in about a second. Everything between submit and the final poll is yours — other work, other jobs, or nothing at all.



Predict and *proba*, async.

Two submits. One shared poll method.

● ● ● module_05_async_patterns.ipynb · cells 13 & 15

```
# async probability prediction
proba_task = nexus_async.submit_predict_proba_task(X_holdout_slim)
proba = nexus_async.poll_predict_result(proba_task)
while proba is None:
    time.sleep(10)
    proba = nexus_async.poll_predict_result(proba_task)
# → np.ndarray shape (n_samples, n_classes)

# async class-label prediction – same poll method
labels_task = nexus_async.submit_predict_task(X_holdout_slim)
labels = nexus_async.poll_predict_result(labels_task)
while labels is None:
    time.sleep(10)
    labels = nexus_async.poll_predict_result(labels_task)
# → np.ndarray of int labels
```

WHY THIS MATTERS

One poll, two results.

Four submit verbs — fit, predict, predict_proba, feature importance — map to three poll methods. predict and predict_proba both retrieve through poll_predict_result: it returns class labels for a predict task and a probability matrix for a predict_proba task.



Two jobs.
One wait.



Submit both. *Poll once.*

N experiments. Wall-clock time of the longest.

● ● ● module_05_async_patterns.ipynb · cells 17 & 18

```
nexus_A = NEXUSClassifier(mode="quality")
nexus_B = NEXUSClassifier(mode="speed")

# submit both before polling either – neither blocks
task_A = nexus_A.submit_fit_task(X_train_slim, y_train)
task_B = nexus_B.submit_fit_task(X_train_full, y_train)

# one shared loop – checks each until done
done = {"A": False, "B": False}
while not all(done.values()):
    if not done["A"] and nexus_A.poll_fit_result(task_A) is not None:
        done["A"] = True
    if not done["B"] and nexus_B.poll_fit_result(task_B) is not None:
        done["B"] = True
    if not all(done.values()):
        time.sleep(20)
# both fitted once the loop exits – no sequencing required
```

WHY THIS MATTERS

The real multiplier.

Sequential, two training runs cost two wall-clocks. Parallel, they cost one. The only ceiling is the number of concurrent jobs your API tier supports — run as many experiments as you can submit.



THE MULTIPLIER

Two jobs. *One* wall-clock.

Sequential training runs cost you *two* wall-clocks. Parallel training — when you submit both jobs before polling either — costs you the *longer* of the two. N experiments for the price of the slowest.

Both jobs run on Fundamental's infra simultaneously; your shared poll loop just checks each. There is no orchestration to write on your side.

A TYPICAL RUN

*~60s each,
still ~60s
in parallel*

Module 05's exercise runs Model A (top-15, quality) and Model B (full, speed). Each takes roughly a minute. In parallel they finish together; sequential, it is twice that — and every added experiment widens the gap.

$N \rightarrow 1$ wait the longest job is the whole budget, no matter how many you submit.



TASK PORTABILITY

Submit at *night*. Poll in the *morning*.

The submission and polling sides share nothing but the task id. A data pipeline at 11 PM submits training and saves the id. An evaluation script at 7 AM loads the id, creates a fresh NEXUSClassifier, and polls — on a different machine, in a different process, with no shared memory.

Once a job completes, its `trained_model_id_` is the permanent handle. Set it on any estimator to reload the model — the same verb you learned in Module 02.

THE LIFETIME

*The task id
is valid for
48 hours*

After that, the job is evicted from the task registry. If you need the model beyond 48h, save the `trained_model_id_` — that one is permanent.

48 h task id lifetime — portable
across sessions and scripts.



TWO IDS, TWO LIFETIMES

Save them *both*. Trust them differently.

`task_id` is the handle on an *in-flight job*. Use it to poll, to resume from another session, to pass between teams. After 48 hours it is evicted from the task registry — the job itself is long finished, but the handle is gone.

`trained_model_id_` is the handle on the *weights*. It lives in the model registry *permanently*. Every future session that wants to predict starts by setting this id on a fresh estimator.

THE OPERATIONAL KEY

task_id:
48 hours
trained_model_id_:
forever

The moment a poll returns, you have both. Save the *trained_model_id_* with care — that is what Modules 02 and 06 reload. The task id has done its job.

48h → ∞ one id fades,
the other stays.



05

*Two jobs.
One wait.
One fallback
note.*

FORMAT

Solo, at your laptop

MATERIALS

Slim and full feature
frames from Module 04

OUTPUT

Two model ids + AUCs,
and a one-paragraph
fallback plan

DIFFICULTY

Intermediate · ●●●○

PARTICIPANT INSTRUCTIONS

Submit two training jobs *in parallel*. Poll them in one shared loop. Then write the fallback plan for when *one of them fails*.

- 01 Submit *model A* on the slim (top-15) frame in quality mode, and *model B* on the full frame in speed mode.
- 02 Poll both in a shared loop. Record each task id and trained_model_id_ as the jobs complete.
- 03 Score the holdout with each model. Note which AUC is higher and by how much.
- 04 Write one paragraph: if one job never completes within an hour, what is your fallback — and why.

Three *takeaways*

If you forget everything else, remember these.

O1

Submit returns *immediately*

`submit_fit_task`, `submit_predict_task`,
`submit_predict_proba_task`. Each returns a
task id in about a second. Match each submit to
its matching poll — don't cross the wires.

O2

Parallel *collapses* wall-clock
time

Submit N before polling any. One shared loop
checks each. N experiments cost the duration of
the longest one, not the sum.

O3

Task ids are *portable strings*

Valid for 48 hours. Save to a file, a database, or
an email. Poll from any estimator, on any
machine — no shared state required.



Next — resilient pipelines.

Module 06 · Exception hierarchy, retries, timeouts · 300-level

ASYNC GAVE YOU SCALE. RESILIENCE KEEPS THE PIPELINE RUNNING WHEN THE NETWORK DOES NOT.